

Socket Programming

Reti di Calcolatori Modulo 2

Using Protocols

- ▶ Protocol software usually embedded in OS
- ▶ Applications run outside OS
- ▶ Need an *Application Program Interface (API)* to allow application to access protocols (TCP/IP)

API

- ▶ TCP/IP standards
 - ▶ Describe general functionalities
 - ▶ Do not give details such as function names and arguments
 - ▶ Do not specify how applications interact with protocols
- ▶ Each OS free to define its own API
- ▶ In practice: *socket interface has become de facto standard API*

Socket API

- ▶ Defined by U.C. Berkeley as part of BSD UNIX
- ▶ Adopted (with minor changes) by Microsoft as *Windows Sockets*
- ▶ Network I/O based on *socket* abstraction
 - ▶ Socket is a generalization of the UNIX file access mechanism

UNIX I/O Paradigm

- ▶ UNIX developed in 1960/70 as a timesharing system
- ▶ Process-oriented operating system
 - ▶ An application program accesses the OS by *system calls*
- ▶ UNIX I/O follows *open-read-write-close* paradigm
 - ▶ User process calls *open* specifying the file and retrieves a *file descriptor*
 - ▶ It calls *read/write* and *close* to inform the end of interaction

UNIX Network I/O

- ▶ Network I/O too complex for Unix's open-read-write-close paradigm
 - ▶ Protocol interface must allow programmers to produce active clients and passive servers
 - ▶ A destination address may be specified for each datagram
 - ▶ Many new system calls and libraries added
- ▶ General mechanism to support many protocols

The Socket Abstraction

- ▶ Basis for network I/O
- ▶ It builds on the *socket* abstraction
- ▶ *Socket is a generalization of the UNIX file access mechanism that provides an endpoint for communication*

Characteristics of Socket API

- ▶ Follows Unix's open-read-write-close paradigm
- ▶ Uses Unix's *descriptor abstraction*
 - ▶ First, create a socket and receive an integer descriptor
 - ▶ Second, call a set of functions that specify all the details for the socket (descriptor is argument to each function)
- ▶ Once a socket has been established, use *read and write* or equivalent functions to transfer data
- ▶ When finished, close the socket

Characteristics of Socket API

- ▶ Main difference
 - ▶ Operating system binds a file descriptor to a file when *open* is called
 - ▶ Operating system can create a socket without binding it to a destination address
- ▶ Read and write used for both file and socket
 - ▶ File and socket descriptors from the same integer set
 - ▶ Operating system manages conflicts

Creating a Socket

- ▶ Argument specifies protocol family as TCP/IP
 - ▶ `sd = socket(pf, type, protocol)`
 - ▶ Protocol family specifies how to interpret addresses (e.g., PF_INET is the TCP/IP internet)
 - ▶ Service type specifies the type of communication (e.g., SOCK_STREAM reliable stream delivery service, SOCK_DGRAM, SOCK_RAW)  TCP  UDP
 - ▶ Protocol to select a specific protocol in case a pair family-type has more protocols 

Creating a Socket

- ▶ Protocol family and service type seem enough to distinguish all possible cases
 - ▶ It may be that some types are not supported by some families
 - ▶ Some families have multiple protocols that supports one type (e.g., two connectionless)
- ▶ The third parameter used to select a specific protocol
 - ▶ The programmer must understand the protocol family

Socket Termination

- ▶ Closing a socket permanently terminates the interaction

└─ Socket descriptor

- ▶ `close(sd)`

- ▶ A call to close decrements a counter and destroy the socket if it is equal to zero

Specifying a Local Address for the Socket (Server)

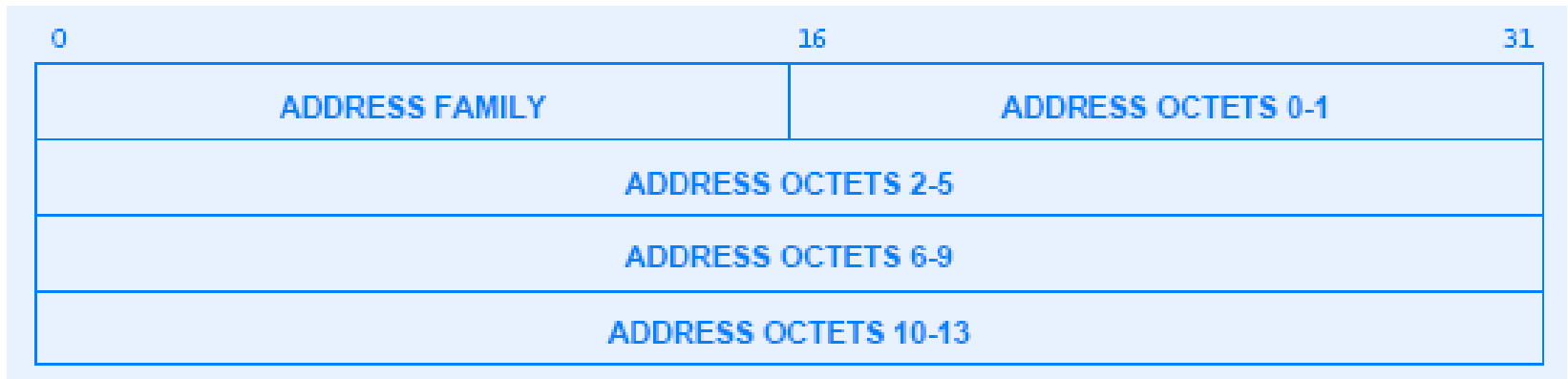
- ▶ Socket initially specified without local or destination address
- ▶ Application programs do not care about local address
- ▶ Server operating at a well-known port should be able to specify that port
- ▶ Specify a local address for the socket
 - ▶ `bind(sd, localaddr, addrlen)`

Numero

Porta del local host

Format of a Sockaddr Structure (Generic)

- ▶ Structure for addresses
- ▶ 16-bit address family that identifies the protocol suite
- ▶ Address of 14 octets at most

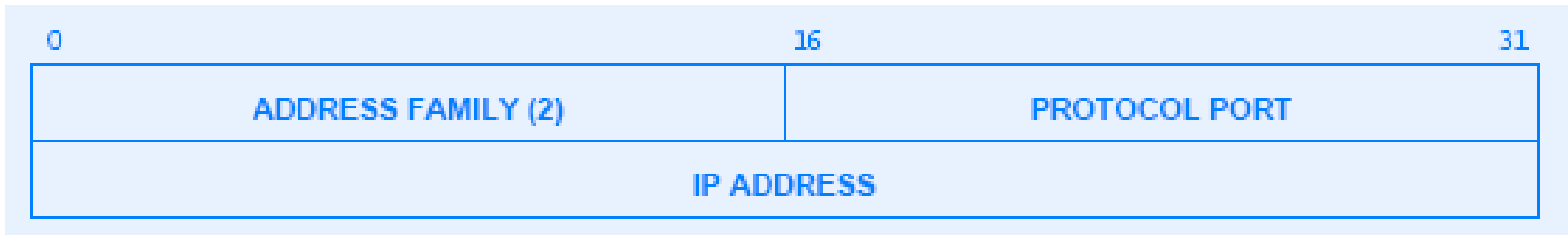


Format of a Sockaddr Structure (Generic)

- ▶ struct sockaddr {
 - u_short sa_family;
 - char sa_data[14];};
- ▶ sa_family
 - ▶ Specify the address family to be used
 - ▶ Determine how the remaining 14 bytes are used

Format of a Sockaddr Structure When Used With TCP/IP

- ▶ ADDRESS FAMILY field establishes the format of the address
 - ▶ Value 2 means TCP/IP address
 - ▶ Sockaddr_in used by socket to specify TCP/IP address



Struct sockaddr_in

- ▶ **struct sockaddr_in {**
 - short sin_family;
 - u_short sin_port;
 - struct in_addr sin_addr;
 - char sin_zero[8];
- };
- ▶ sin_family: AF_INET
- ▶ sin_port: port number (0-65.535)
- ▶ sin_addr: IP address
- ▶ sin_zero: unused

Format of a Sockaddr Structure When Used With TCP/IP

- ▶ Not all possible bindings are correct
 - ▶ Port already in use
 - ▶ Invalid bind address
- ▶ In these cases, the bind call fails and returns an error

Queue Length (Server)

- ▶ Set Maximum Queue Length (Server)
 - ▶ `listen(sd, qlength)` → *Si applica solo a connessione off-socket (TCP)*
 - ▶ Maximum queue length can be quite small
 - ▶ Avoid request rejection
 - ▶ Requests are queued until server has time to process it
 - ▶ Listen applies to reliable stream delivery service
- ▶ Reserved for SOCK_STREAM

Accepting New Connections (Server)

- ▶ Identifies the connection coming from client
- ▶ Server uses **socket**, **bind**, and **listen**, but no connection is established
- ▶ **newsd = accept(sd, addr, addrlen)**
 - ▶ Server waits for connections (wildcard foreign destination)
 - ▶ Original socket remains available for accepting connections
 - ▶ New socket corresponds to one connection
 - ▶ Permits server to handle requests concurrently
 - ▶ New socket descriptor sent back to the caller
- ▶ Reserved for SOCK_STREAM

Socket di ascolto (master)

Indirizzo del client

Accepting New Connections (Server)

▶ Iterative approach

- ▶ Server handles the request
- ▶ Server closes the new socket
- ▶ Server calls *accept*

→ struttura la socket listening

▶ Concurrent approach

- ▶ Server creates a slave
- ▶ Slave handles the request
- ▶ Server closes its copy of the new socket and calls *accept*

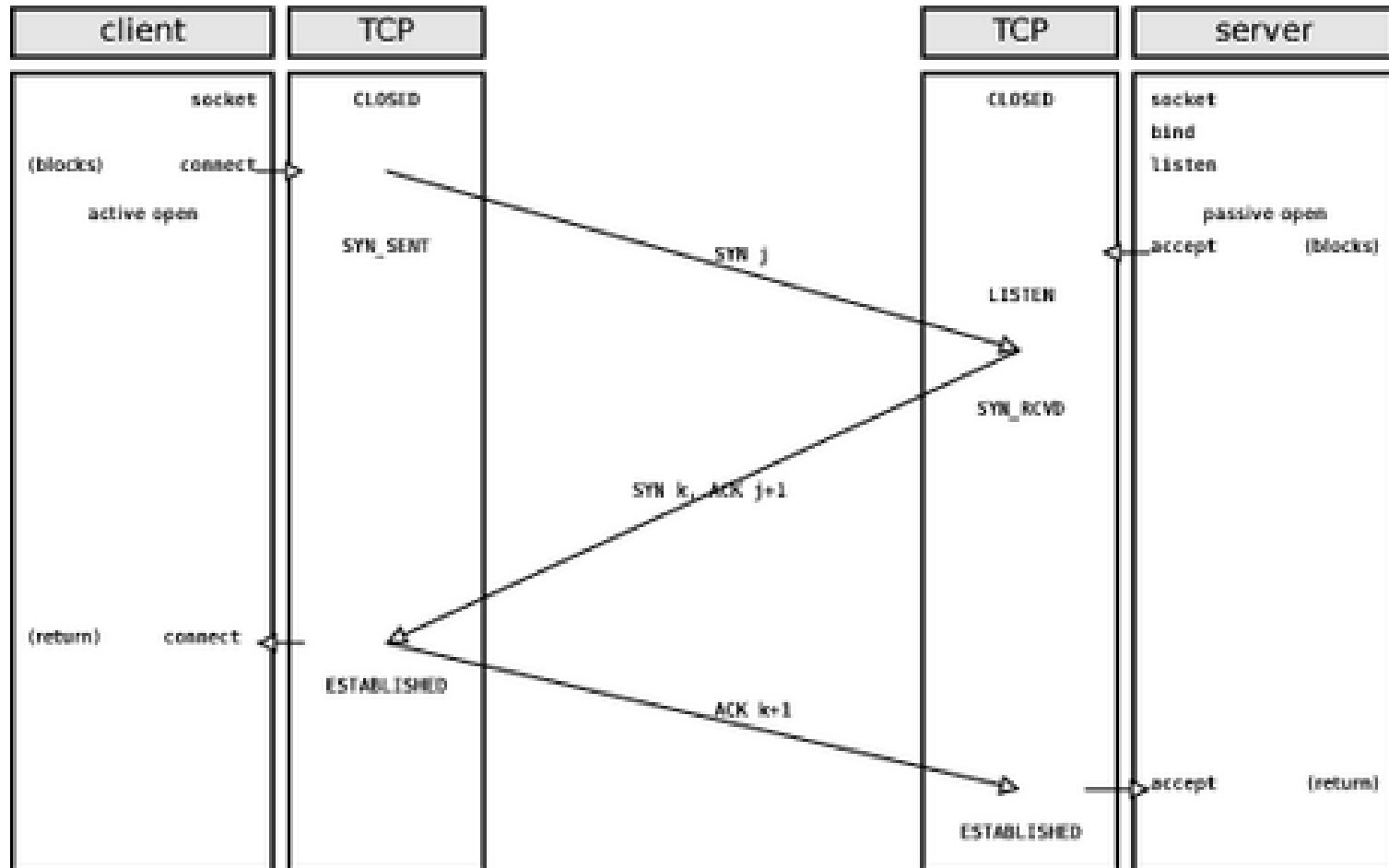
Concurrent Approach

- ▶ Each server slave manages the communication for a pair of endpoints (server address is the same)
- ▶ Server master manages connection requests which do not have a process slave associated *→ wildcard*
- ▶ For each TCP segment, a slave is searched
 - ▶ If it is not found, request forwarded to the master

Connecting a Socket to a Destination Address

- ▶ Socket initially in *unconnected* state
- ▶ Connect can be used to specify remote endpoint address (connected state)
 - ▶ `connect(socket, destaddr, addrlen)`
 - ↳ Questo socket è nel OS del client
 - ↳ Indirizzo del server
 - ↳ Lanciata dal client
 - ↳ duali di accept per server
- ▶ Semantic of *connect* depends on the family protocol
 - ▶ PF_INET: *connect* builds a TCP connection
- ▶ Connectionless datagram services do not use connect function → eg. UDP
- ▶ Reserved for SOCK_STREAM

Socket and three-way handshake



Sending Data Through a Socket

- ▶ Application can use socket to send data

↳ In UDP serve aggiungere indirizzo del destinatario → non c'è funz. connect che stabilisca la connessione → vedi prossima slide

- ▶ Function for **connected sockets** (SOCK_STREAM)

↳ TCP

- ▶ **write(sd, buffer, length)**: message is in a buffer
- ▶ **writen(sd, iovec, vectorlen)**: message is **not in contiguous bytes of memory**. iovec refers to array **iovec of pointers**, **vectorlen** is the **number of entries in iovec**
- ▶ **send(sd, message, length, flags)**: flags control the transmissions (e.g., message to be sent out of band)

Sending Data Through a Socket

- ▶ Function for ^{↖ UDP}unconnected sockets (SOCK_DGRAM)
 - ▶ `sendto(sd, message, length, flags, destaddr, addrlen)`: the same of *send* function except for the last two arguments
 - ▶ `sendmsg(sd, messagestruct, flags)`: useful since there is an input operation using the structure of messagestruct
 - ▶ Messagestruct contains the message (iovector), address, access rights
- ▶ All functions must specify the destination

Receiving Data Through a Socket

- ▶ Socket API provides five functions to receive data through a socket
- ▶ Function for connected sockets (SOCK_STREAM)
 - ▶ `read(sd, buffer, length)`
 - ▶ `readv(sd, iovector, vectorlen)`
 - ▶ `recv(sd, buffer, length, flags)`
- ▶ Function for unconnected sockets (SOCK_DGRAM)
 - ▶ `recvfrom(sd, buffer, length, flags, fromaddr, addrlen)`
 - ▶ `Fromaddr` and `addrlen` used to store the address of the message sender
 - ▶ `recvmsg(sd, messagestructs, flags)`

Obtaining Remote and Local Socket Addresses

- ▶ New processes created using `fork` inherits the set of open sockets from the parents
- ▶ Sometimes new processes need to know destination and local addresses
 - ▶ `getpeername(sd, destaddr, addrlen)`
 - ▶ Retrieve the address of the remote peer
 - ▶ Work with connected socket $\rightarrow$ TCP
 - ▶ `getsockname(sd, localaddr, addrlen)`
 - ▶ Return local address of the socket

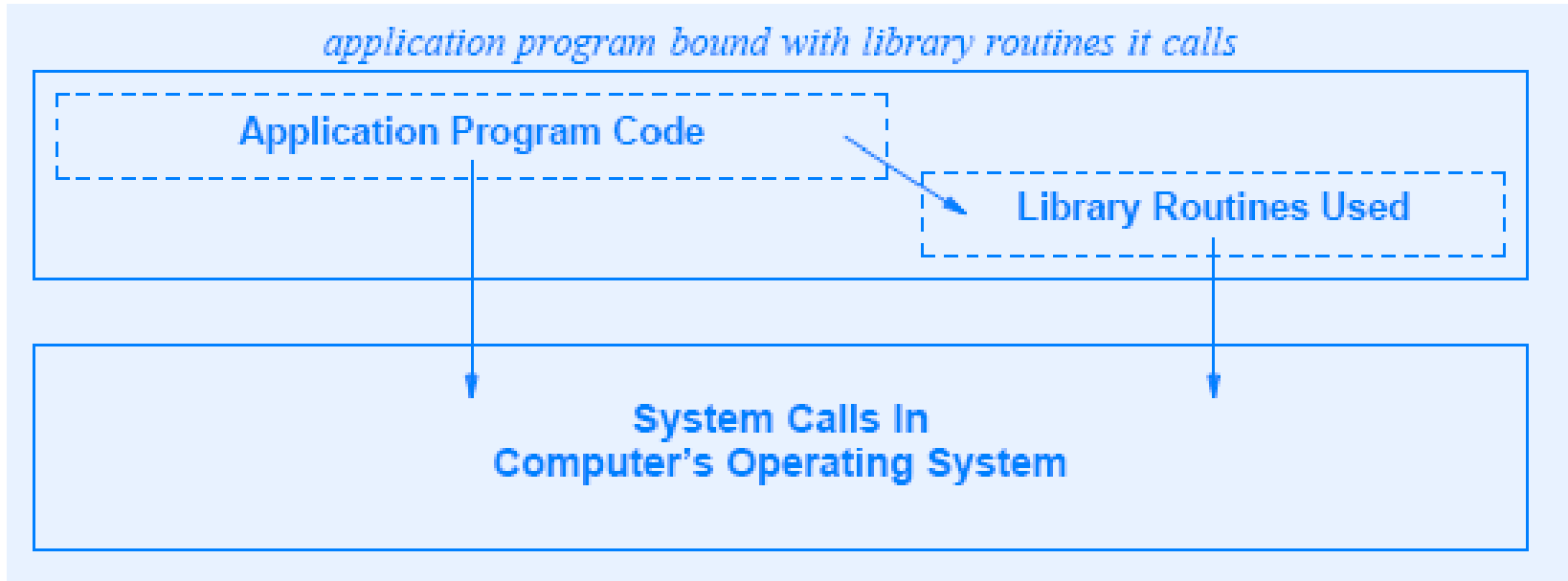
Socket Options

- ▶ Need of a mechanism to control the socket
 - ▶ Set the *timeout* for protocols using timeout and retransmission
 - ▶ Control allocation of the buffer space
 - ▶ Determine if broadcast is allowed
 - ▶ Control processing of out-of-band data
- ▶ As for SNMP, no new function for each option, but a single mechanism

Socket Options

- ▶ Function *getsockopt* and *setsockopt* used
 - ▶ *getsockopt(sd, level, optionid, optionval, lenght)*
 - ▶ Application requests information about the socket
 - ▶ OS accesses internal data structures and returns a response
 - ▶ *setsockopt(sd, level, optionid, optionval, lenght)*
 - ▶ Application sets a socket option
 - ▶ The caller specifies a socket, the option and the value
- ▶ Not all options apply to all sockets

Illustration Of A Socket Library



- ▶ Additional functions
- ▶ Provide functions related to networking
- ▶ Mostly provide database services (e.g., determine the name of the machine, the port number, protocol service)

Byte Order Conversion Routines

- ▶ Machines and OSs use different integer representation
 - Little-endian: less-significant bytes are stored first
 - Big-endian (network ordering): most-significant bytes are stored first
- ▶ An independent network representation is provided
 - ↳ Big-endian
- ▶ Need to convert between network byte order and local host byte order, and viceversa

Byte Order Conversion Routines

- ▶ If local host uses big-endian, routines have no effect
- ▶ Conversion functions for short (2 bytes) and long (4 bytes) integer
 - ▶ `localshort = ntohs(netshort)` → network to host → Big endian to little endian
 - ▶ `locallong = ntohl(netlong)` → network to host → Big endian to little endian
 - ▶ `netshort = htons(localshort)` → host to network → Little endian to Big endian
 - ▶ `netlong = htonl(locallong)` → host to network → Little endian to Big endian

Byte Order Conversion Routines

- ▶ Conversion routines must satisfy the following mathematical relationships
 - ▶ `netshort = htons(ntohs(netshort))`
 - ▶ `localshort = ntohs(htons(localshort))`
 - ▶ The same for long integer

IP Address Manipulation Routines

- ▶ Convert from dotted decimal (ASCII string) to 32-bit binary value in network byte order
 - ▶ `address = inet_addr(string)` or `inet_aton(string,addr)`
 - ▶ `address = inet_network(string)`
- ▶ Inverse of `inet_addr`
 - ▶ `str = inet_ntoa(internetaddr)`

IP Address Manipulation Routines

- ▶ Combine a network address with the local address of the host on that network
 - ▶ `internetaddr = inet_makeaddr(net, local)`
- ▶ Inverse of `inet_makeaddr`
 - ▶ `net = inet_netof(internetaddr)`
 - ▶ `local = inet_lnaof(internetaddr)`

Byte Order Conversion Example: IP Address and Port

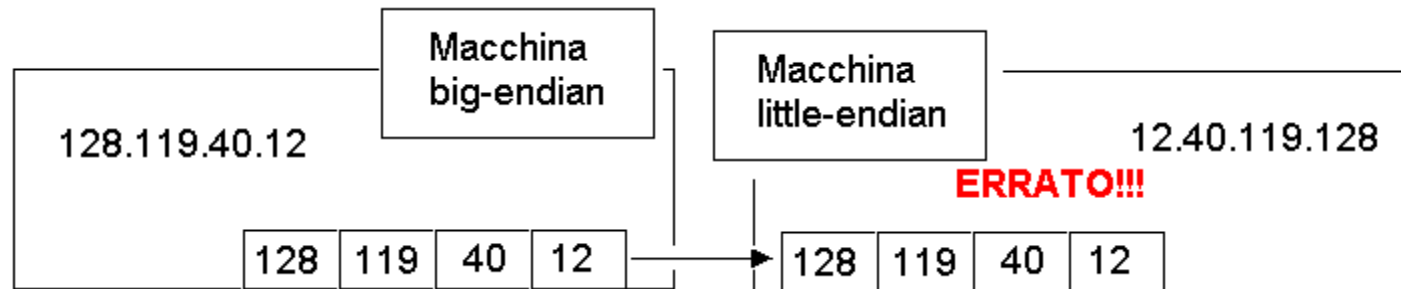
- ▶ IP address and port store as integers
 - u_short sin_port; (16 bit)
 - in_addr sin_addr; (32 bit)

How can different machine communicate over the network?

Byte Order Conversion Example: IP Address and Port

- ▶ IP address and port store as integers
 - u_short sin_port; (16 bit)
 - in_addr sin_addr; (32 bit)

How can different machine communicate over the network?

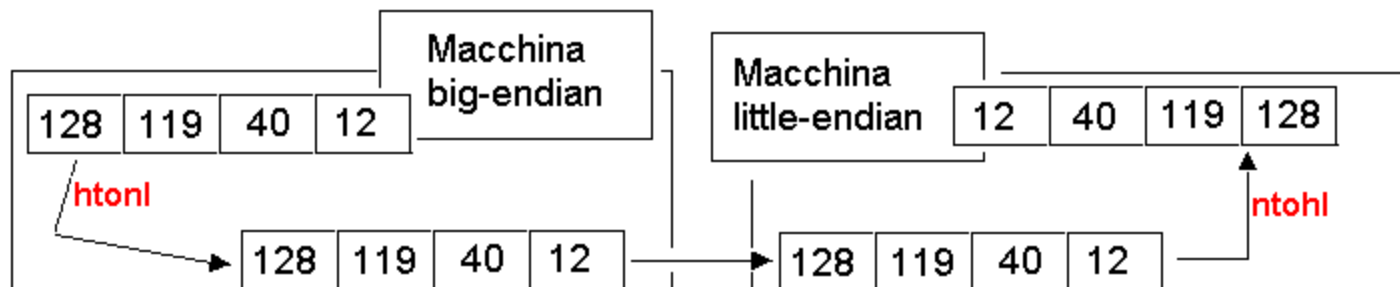


Byte Order Conversion Example: IP Address and Port

- ▶ Host byte ordering
 - ▶ Big endian or little-endian
- ▶ Network byte ordering
 - ▶ Big-endian
- ▶ Before sending a word through the network it must be converted in the network byte ordering
- ▶ When received it must be converted back to the host byte ordering

Byte Order Conversion Example: IP Address and Port

- ▶ `u_long htonl(u_long x);`
 - ▶ `u_short htons(u_short x);`
 - ▶ `u_long ntohl(u_long x);`
 - ▶ `u_short ntohs(u_short x);`
- ▶ On big-endian machines, these procedures have no effect
 - ▶ On little-endian machines, these procedures change the byte ordering



- ▶ The same code works independently by the hosts byte ordering

Socket Library

- ▶ Different groups of library available
 - ▶ Many of them provide access to databases
 - ▶ Allow a process to retrieve information
- ▶ A pattern of three routines is provided for sequential access to the database
 - ▶ setXent: to establish a connection
 - ▶ getXent: to access some information
 - ▶ endXent: to close a connection
- ▶ Example for the host database
 - ▶ sethostent, gethostent, endhostent

Sequential Access

- ▶ In all libraries, routines used for sequential access to the database are specified
 - ▶ Information about host: `sethostent`, `gethostent`, `endhostent`
 - ▶ Information about network: `setnetent`, `getnetent`, `endnetent`
 - ▶ Information about protocol: `setprotoent`, `getprotoent`, `endprotoent`
 - ▶ Information about service: `setservent`, `getservent`, `endservent`

Socket Functions Used For DNS

- ▶ Get the internal host name maintained by an operating system
 - ▶ `gethostname(name, length)`
- ▶ Set the internal host name maintained by an operating system
 - ▶ `sethostname(name, length)`
- ▶ Obtaining the local domain
 - ▶ `getdomainname(name, length)`
- ▶ Setting the local domain
 - ▶ `setdomainname(name, length)`

Accessing the Domain Name System

- ▶ Library procedures to interface with the DNS
- ▶ The application program makes a query and waits for a response
- ▶ The procedures have a few basic parameters and a global structure *res* to manage all options (e.g., debug, protocol)

Accessing the Domain Name System

- ▶ First, `res_init()` is called
 - ▶ It reads a file with the name of the DNS machine and stores the result in `res`
- ▶ Then, `res_mkquery`
(`op,dname,class,type,data,datalen,newrr,buffer,buflen`)
forms a domain query
- ▶ Finally, `res_send(buffer, buflen,answer, anslen)` sends the query to the DNS

Other Socket Routines

- ▶ Many other functions exist
- ▶ Examples: obtain information about
 - ▶ Hosts
 - ▶ Domain name
 - ▶ Protocols

Information about Hosts

- ▶ A process can retrieve information about a host given its domain name or IP address
- ▶ When a DNS exists, the library procedures send a request to the server, otherwise access a local database
 - ▶ `ptr = gethostbyname(namestr)` retrieves information by host name
 - ▶ `ptr = gethostbyaddr(addr, len, type)` retrieves information by host address
 - ▶ `ptr` includes official host name, list of aliases, address type, address length, one or more addresses

Information about Networks

- ▶ A process retrieves information about the network by accessing the DNS or the network database
 - ▶ `ptr = getnetbyname(name)` retrieves information by network name
 - ▶ `ptr = getnetbyaddr(netaddr, addrtype)` retrieves information by network address
 - ▶ `ptr` includes official network name, list of aliases, address type, 32-bit network address

Information About Protocols and Network Services

- ▶ Same happens for protocols
 - ▶ `ptr = getprotobyname(name)`
 - ▶ `ptr = getprotobyname(number)`
 - ▶ `ptr` contains protocol name, aliases, integer value
- ▶ and for network services
 - ▶ `ptr = getservbyname(name,proto)`
 - ▶ `ptr = getservbynumber(port,proto)`
 - ▶ `ptr` contains service name, aliases, protocol identifier, port number
 - ▶ E.g., `getservbyname("whois",tcp)`
 - ▶ Whois, tcp, 43

Example: Whois Service

- ▶ Implementation of whois client and server
 - ▶ Allows a client on one machine to receive information about a user on a remote system
- ▶ Shows how a program uses the socket API to access TCP/IP protocols
- ▶ The client maps the server to the real address, creates the socket and communicates with the server

Example: Client (C)

- ▶ The client is an application program
 - ▶ Name of the remote server and the name of the remote user passed as arguments
- ▶ The client calls
 - ▶ *gethostbyname* to map the remote server name to an IP address
 - ▶ *getservbyname* to retrieve the well-known port of the Whois service
- ▶ The client creates a socket using TCP and bind it to the well-known port on the remote machine

Example: Client (C)

```
/* whoisclient.c - main */
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <string.h>
#include <stdlib.h>

/*-----
 * Program: whoisclient
 *
 * Purpose: UNIX application program that becomes a client for the
 * Internet "whois" service.
 *
 * Use: whois hostname username
 *
 * Author: Barry Shein, Boston University
 *
 * Date: Long ago in a universe far, far away
 *-----
 */
```



Example: Client (C)

```
main(argc, argv)
int argc; /* standard UNIX argument declarations */
char *argv[];
{
    int s; /* socket descriptor */
    int len; /* length of received data */
    struct sockaddr_in sa; /* Internet socket address structure */
    struct hostent *hp; /* result of host name lookup */
    struct servent *sp; /* result of service lookup */
    char buf[BUFSIZ+1]; /* buffer to read whois information */
    char *myname; /* pointer to name of this program */
    char *host; /* pointer to remote host name */
    char *user; /* pointer to remote user name */
    myname = argv[0];
```

Example: Client (C)

```
/* Check that there are two command line arguments */
if(argc != 3) {
    fprintf(stderr, "Usage: %s host username\n", myname);
    exit(1);
}
host = argv[1];
user = argv[2];
/* Look up the specified hostname */
if((hp = gethostbyname(host)) == NULL) {
    fprintf(stderr, "%s: %s: no such host?\n", myname, host);
    exit(1);
}
/* Put host's address and address type into socket structure */
bcopy((char *)hp->h_addr, (char *)&sa.sin_addr, hp->h_length);
sa.sin_family = hp->h_addrtype;
```

Example: Client (C)

```
/*Look up the socket number for the WHOIS service*/
if((sp = getservbyname("whois","tcp")) == NULL) {
    fprintf(stderr,"%s: No whois service on this host\n", myname);
    exit(1);
}
/* Put the whois socket number into the socket structure. */
sa.sin_port = sp->s_port;
/* Allocate an open socket */
if((s = socket(hp->h_addrtype, SOCK_STREAM, 0)) < 0) {
    perror("socket");
    exit(1);
}
```

Example: Client (C)

```
/* Connect to the remote server */
if(connect(s, &sa, sizeof sa) < 0) {
    perror("connect");
    exit(1);
}
/* Send the request */
if(write(s, user, strlen(user)) != strlen(user)) {
    fprintf(stderr, "%s: write error\n", myname);
    exit(1);
}
/* Read the reply and put to user's output */
while( (len = read(s, buf, BUFSIZ)) > 0)
    write(1, buf, len);
close(s); → Client non è persistente
exit(0);
}
```


Example: Server (C)

- ▶ More complex than the client
- ▶ The server listens on the well-known whois port (43)
- ▶ The server returns the information requested by the client

Example: Server (C)

```
/* whoisserver.c - main */
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <pwd.h>
#include <string.h>
#include <stdlib.h>

/*-----
* Program: whoisserver
*
* Purpose: UNIX application program that acts as a server for
* the "whois" service on the local machine. It listens
* on well-known WHOIS port (43) and answers queries from
* clients. This program requires super-user privilege to
* run.
*
* Use: whois hostname username
*
*/
```



Example: Server (C)

```
* Author: Barry Shein, Boston University
*
* Date: Long ago in a universe far, far away
*
* -----
*/
#define BACKLOG 5 /* # of requests we're willing to queue */
#define MAXHOSTNAME 32 /* maximum host name length we tolerate */
main(argc, argv)
int argc; /* standard UNIX argument declarations */
char *argv[];
{
    int s, t; /* socket descriptors */
    int i; /* general purpose integer */
    struct sockaddr_in sa, isa; /* Internet socket address structure */
    struct hostent *hp; /* result of host name lookup */
    char *myname; /* pointer to name of this program */
    struct servent *sp; /* result of service lookup */
    char localhost[MAXHOSTNAME+1]; /* local host name as character string */
```



Example: Server (C)

```
myname = argv[0];
/* Look up the WHOIS service entry */
if((sp = getservbyname("whois","tcp")) == NULL) {
    fprintf(stderr, "%s: No whois service on this host\n", myname);
    exit(1);
}
/* Get our own host information */
gethostname(localhost, MAXHOSTNAME);
if((hp = gethostbyname(localhost)) == NULL) {
    fprintf(stderr, "%s: cannot get local host info?\n", myname);
    exit(1);
}
```

Example: Server (C)

```
/* Put the WHOIS socket number and our address info into the socket
   structure*/
sa.sin_port = sp->s_port;
bcopy((char *)hp->h_addr, (char *)&sa.sin_addr, hp->h_length);
sa.sin_family = hp->h_addrtype;
/* Allocate an open socket for incoming connections */
if((s = socket(hp->h_addrtype, SOCK_STREAM, 0)) < 0) {
    perror("socket");
    exit(1);
}
/* Bind the socket to the service port so we hear incoming connections */
if(bind(s, &sa, sizeof sa) < 0) {
    perror("bind");
    exit(1);
}
```

Example: Server (C)

```
/* Set maximum connections we will fall behind */
listen(s, BACKLOG);
/* Go into an infinite loop waiting for new connections */
while(1) {
    i = sizeof isa;
    /* We hang in accept() while waiting for new customers */
    if((t = accept(s, &isa, &i)) < 0) {
        perror("accept");
        exit(1);
    }
    whois(t); /* perform the actual WHOIS service */
    close(t);
}
}
```

Example: Server (C)

```
/* Get the WHOIS request from remote host and format a reply. */
whois(sock)
int sock;
{
    struct passwd *p;
    char buf[BUFSIZ+1];
    int i;

    /* Get one line request */

    if( (i = read(sock, buf, BUFSIZ)) <= 0)
        return;
    buf[i] = '\0'; /* Null terminate */
}
```

Example: Server (C)

```
/* Look up the requested user and format reply */  
if((p = getpwnam(buf)) == NULL)  
    strcpy(buf,"User not found\n");  
else  
    sprintf(buf, "%s: %s\n", p->pw_name, p->pw_gecos);  
  
/* Return reply*/  
  
write(sock, buf, strlen(buf));  
return;  
}
```


Exercise 1

- ▶ Starting from the code available in ARIEL providing a client and a server UDP implementing an Echo server, implement a client and a server TCP, with **iterative TCP socket**, which provide the same Echo service
 - ▶ The client sends a string to the server
 - ▶ The server returns the same string
- ▶ Comment the code underlying the most significant parts of the TCP connection

Exercise 1 – Bind (Error)

- ▶ Upon developing a working client server try the following actions:
 - ▶ Start two servers on the same port 3333
 - ▶ Start a server on port 3333, open the client, and connect it to the server
 - ▶ Close the server using CTRL+C
 - ▶ Start a server on port 3333

Exercise 1 – Bind (EADDRINUSE)

- ▶ Upon developing a working client server try the following actions:
 - ▶ Start two servers on the same port 3333
 - ▶ **ERROR (address already in use - EADDRINUSE): the port is still in use by another process**
- ▶ Start a server on port 3333, open the client, and connect it to the server
 - ▶ Close the server using CTRL+C
 - ▶ Start a server on port 3333
- ▶ **ERROR (address already in use - EADDRINUSE): operation close(sd) is not call properly (see more details in the following slides)**

Exercise 1 – Bind (Error)

- ▶ **ERROR: operation close(sd) is not call properly (see more details in the following slides)**
- ▶ Possible solutions
 - ▶ Catch sigint CTRL+C with sighandle or sigaction, and close the socket correctly
 - ▶ Use option configurations
 - ▶ if (setsockopt(socketdescriptor, SOL_SOCKET, SO_REUSEADDR, &(int){1}, sizeof(int)) < 0)
perror("setsockopt(SO_REUSEADDR) failed");
 - ▶ if (setsockopt(socketdescriptor, SOL_SOCKET, SO_REUSEPORT, &(int){1}, sizeof(int)) < 0)
perror("setsockopt(SO_REUSEPORT) failed");

Exercise 2

- ▶ Implement a distributed software (server+client) in C language, with **iterative TCP socket**, which provides a service that concatenates two strings
 - ▶ The client sends two strings
 - ▶ The server returns the concat of the two strings
- ▶ Comment the code underlying the most significant parts of the TCP connection
- ▶ To implement the concat service, you can use the following C functions
 - ▶ strcpy / strncpy
 - ▶ strcat / strncat

Exercise 3

- ▶ Implement a distributed software (server+client) in C language, with **iterative TCP socket**, which provides a service that receive as input an integer of a distance in km and return an integer of the same distance in m
- ▶ Comment the code underlying the most significant parts of the TCP connection
- ▶ To implement the service, use the following C functions for endianness
 - ▶ htonl
 - ▶ ntohl

Parallel Socket: fork

- ▶ Duplicate the process and return
 - ▶ To the parent or caller: pid (Process Id) of the child process
 - ▶ To the child: pid = 0
 - ▶ In case of error: -1
- ▶ Open descriptors of the parent process will be shared by the child process
- ▶ A closed descriptor in a process remains open in the other process
- ▶ Goal: Create replicas of the socket server process to manage parallel communications
- ▶ The execution starts the next instruction following the fork() system call
- ▶ System call for Linux systems, unavailable under Windows systems

↳ Sol. variante if su PID

```
#include <unistd.h>
```

```
pid_t fork (void);
```

Parallel Socket: Example

```
pid_t pid;
int listenfd, connfd;

listenfd = socket (AF_INET, SOCK_STREAM, 0);
bind ( listenfd, (struct sockaddr*) &Local, sizeof(Local));
listen(listenfd, 10 );
for( ; ; ) {
    connfd = accept ( listenfd, (struct sockaddr*) &Cli, &len);
    pid = fork();

    if ( pid !=0)
        close(connfd);

    else {
        close(listenfd);
        handleConnection(connfd);
        close(connfd);
        exit(0);
    }
}
```

Parent process, close the connected socket and return to accept() function

Child process, close the master socket and manage the communication with the connected socket

Socket inheritance and termination

- ▶ When a program calls `fork()`, the new copy (child process) inherits access to all sockets
- ▶ When the program call `exec()`, the new process maintains access to all sockets
- ▶ Socket inheritance is used by a master to create a slave
- ▶ Closing a socket permanently terminates the communication
 - ▶ `close(socket)`
 - ▶ A single call to `close()` decrements the counter and destroy the socket iff such a counter is equal to zero

Exercise 3

- ▶ Implement a distributed software (server+client) in C language, with **parallel TCP socket based on the function `fork()`**, which provides a parallel Echo service
 - ▶ The client sends a string
 - ▶ The server returns the same string
- ▶ Comment the code underlying the most significant parts of the TCP connection

Blocking calls

- ▶ Many of the presented functions block the caller until an event happens
 - ▶ accept: blocks until a connection is received
 - ▶ connect: blocks until a connection is established
 - ▶ recv, recvfrom: blocks until a data packet is received
 - ▶ send, sendto: until the data are stored in the buffer of the socket call
- ▶ For simple programs is useful
- ▶ For complex programs it might be a problem
 - ▶ Multiple connections
 - ▶ Simultaneous sending and connections

Non-blocking calls

- ▶ Multiple options
 - ▶ Use blocking calls, but write multi-thread o multi-process code
 - ▶ Disactive the block option, for instance using the file descriptor control function `fcntl`
 - ▶ `Select()` socket [More on this later]

Non-blocking calls: fcntl

- ▶ The "non-blocking" mode is set by changing one of the socket's "flags"
- ▶ The flags are a series of bits, each one representing a different capability of the socket. So, to turn on non-blocking mode requires three steps:
 - ▶ Call the fcntl() API to retrieve the socket descriptor's current flag settings into a local variable
 - ▶ In our local variable, set the O_NONBLOCK (non-blocking) flag on
 - ▶ Call the fcntl() API to set the flags for the descriptor to the value in our local variable

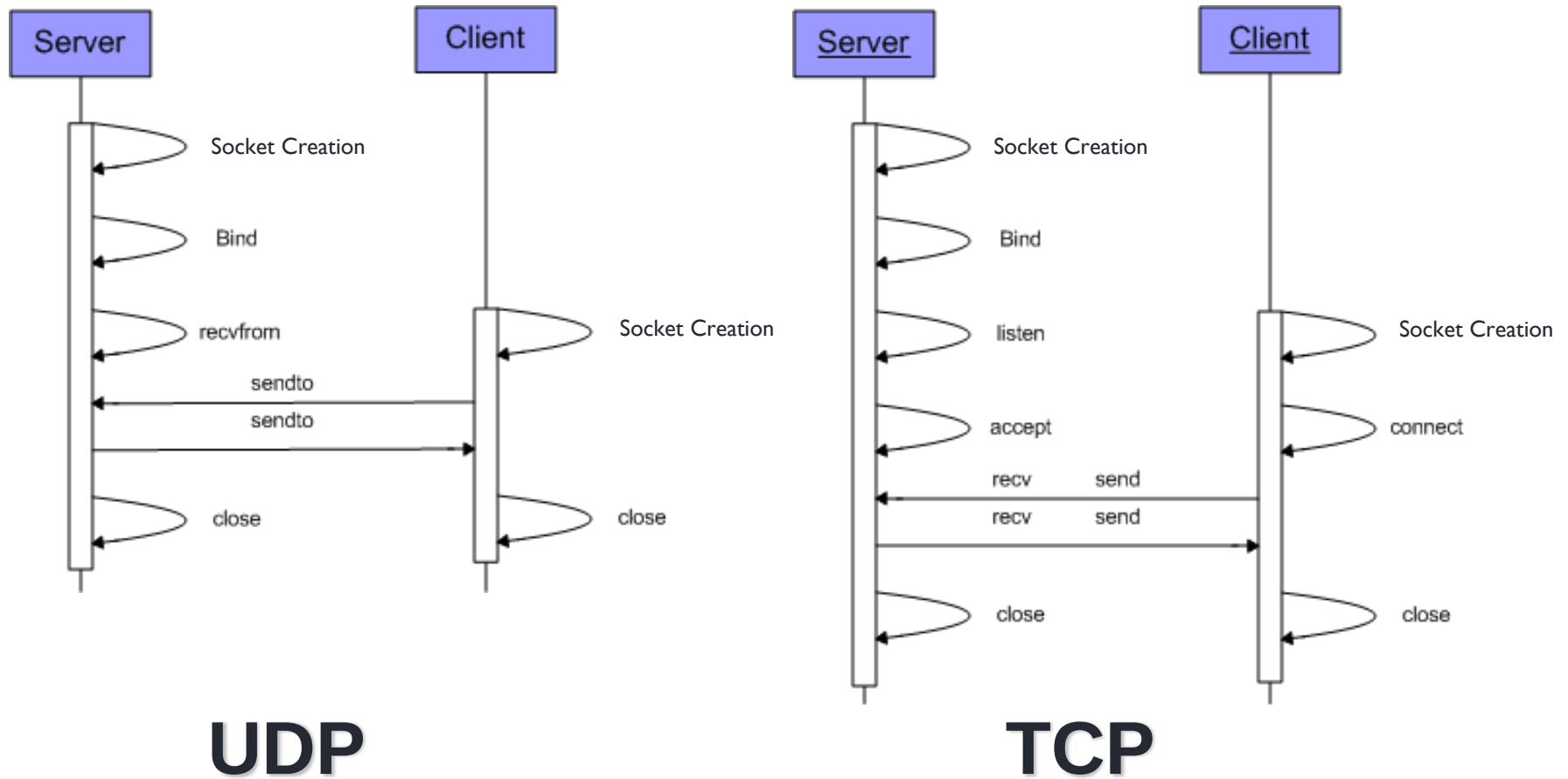
```
// where sockfd is the socket you want to make non-blocking  
int status = fcntl(sockfd, F_SETFL, fcntl(sockfd, F_GETFL, 0) | O_NONBLOCK);
```

```
// simplified version  
fcntl(accept_sd, F_SETFL, O_NONBLOCK);
```

Exercise 4

- ▶ Starting from the code of exercise 1, make the socket slave (the one created by `accept()`) non-blocking. Every time a function returns print «no operation».
- ▶ Comment the code underlying the most significant parts of the TCP connection

In a nutshell



Summary

- ▶ TCP/IP protocol software resides inside OS
- ▶ Exact interface between application program and TCP/IP protocols depends on the OS (not specified by TCP/IP protocol standard)
- ▶ Socket API becomes a de facto standard

Summary

- ▶ Socket API
 - ▶ Invented for BSD Unix
 - ▶ Officially, it is not part of TCP/IP
 - ▶ Adopted the UNIX *open-read-write-close* paradigm
 - ▶ Used with TCP or UDP
 - ▶ Large set of functions
- ▶ General paradigm: create socket and then use a set of functions to specify details

